

用于 PageIndex 检索的存储引擎: MongoDB、PostgreSQL、DuckDB 与 SQLite 的工作负载基准

Junyao Dong

摘要。 PageIndex 把文档变成大型分层 JSON 树,智能体从根节点向下导航到相关节点。我们在 ConDB 检索器对存储发出的确切操作上,对四个存储引擎(MongoDB、PostgreSQL/JSONB、DuckDB,以及 ConDB 当前引擎 SQLite)做基准测试,规模最高达一千万节点。在导航读取、内容获取、写入、并发与存储占用各方面,MongoDB 在除一项之外的每个操作上都有竞争力:获取大子树(`get_subtree`),其尾延迟达到秒级,比关系型引擎差一个数量级以上。我们把它追溯到 MongoDB 的逐文档读取模型,并给出一个优化方案 仅靠索引与 schema 设计,就能在社区版上消除这一差距,无需改动引擎。

1 引言

ConDB 把一篇文档索引成一棵 PageIndex 树:节点构成的层级结构,每个节点带有标题、摘要、页码索引,叶子节点还带有正文。检索是一次自顶向下的遍历:检索器从根节点出发,列出某个节点的子节点以决定向哪里下探,拉取一棵子树供语言模型阅读,再获取它所选中节点的内容。本项目要回答的问题是:文档型存储(MongoDB)是否适合存放这类数据;如果适合,该优化什么。本报告用实测数据和一份具体方案回答这两点。

我们刻意只对 ConDB 实际发出的操作做基准。检索路径从不执行的通用数据库操作(从节点回溯到根、页码区间过滤、朴素子串扫描)被排除在外,因为它们的结果无助于优化。

2 工作负载与方法

操作

两个检索器(`beam_retriever.py`、`block_retriever.py`)通过四个调用触达存储(表

1)。下文测量的一切都是这四者之一,外加存储占用、写入路径(增量索引与推理轨迹的编辑),以及并发。

操作	存储调用	用途
点查	<code>get_node(tree_id, node_id)</code>	导航到 / 读取单个节点
展开子节点	<code>get_children(tree_id, node_id)</code>	列出子节点,决定下探方向
取子树	<code>get_subtree(tree_id, node_id, depth)</code>	渲染树视图、拉取一个 block
取内容	<code>get_entity(tree_id, node_id)</code>	读取所选节点的内容

表 1:ConDB 检索发出的存储操作。

数据

采用规范格式的合成 PageIndex 树,并用 ConDB 的 `DocumentTreeAdapter` 校验,两种规模(表 2)。内容是随机词;基准考察的是树的形状与每节点的负载大小。

数据集	节点数	JSON 大小	深度	扇出
中(medium)	70,843	85 MB	5	6-12
大(large)	10,000,000	14.06 GB	8	6-14

表 2:数据集。

引擎与环境

MongoDB 7.0(社区版)与 PostgreSQL 16(JSONB 文档列)以服务端形式跑在 Docker 中、经 localhost 访问;DuckDB 与 SQLite 为嵌入式。四者跑在同一台 96 核 Linux 主机、1 TB 内存上。每个引擎都以一行/文档对应一个节点的方式存储,字段逻辑一致,并为各查询建好所需索引。树被压平成一行一节点;`get_subt`

ree 用物化路径加字节序区间谓词,四引擎完全一致(PostgreSQL 的 path 列用 COLLATE "C",使其区间与其它引擎的二进制比较一致)。子树查询只返回节点 id;若再带上树视图同样要渲染的标题与摘要,会让关系型引擎更费、却不影响 MongoDB(它无论投影什么都物化整份文档),所以这个选择对 MongoDB 偏保守。每个读取在 500 个采样节点上测量(200 个子树根,采自深度 ≤ 3),由随机种子固定、四引擎一致;每个查询都核对过在四引擎上返回相同的行数。每个并发档位让其客户端进程运行五秒。主机内存充裕,所以每个引擎的数据都常驻缓存,读取不受磁盘约束。

公平性说明

SQLite 与 DuckDB 在进程内运行,一次查询就是一次函数调用;MongoDB 与 PostgreSQL 要付每次调用的客户端-服务端开销(一次 localhost 往返加协议处理),嵌入式引擎从不承担。因此公平的单次调用比较是 MongoDB 对 PostgreSQL;跨越嵌入式/服务端这条线的绝对微秒数不可比,而并发一节(各自独立进程、无 GIL)展示的是真实的服务端吞吐。MongoDB 的 storageSize 是压缩后的(WiredTiger snappy),其它是未压缩的磁盘文件,所以 MongoDB 的未压缩逻辑大小单独列出;且随机词内容大概比真实散文更易压缩,使压缩后的数字偏好看。所有测量都在单主机上完成;到并发为止的各项用一库一树,多租户、批量读取与删除在下文用 25 棵共存树单独测量。

3 结果

3.1 存储与摄入

引擎	磁盘合计	索引	未压缩	摄入(节点/秒)	建表
DuckDB	5.02 GB	0	—	348,831	10 s
MongoDB	5.81 GB	536 MB	15.21 GB	41,387	69 s
PostgreSQL (JSONB)	17.71 GB	1.19 GB	—	62,816	52 s
SQLite	19.51 GB	980 MB	—	52,880	38 s

表 3:存储与摄入(大数据集)。磁盘合计含索引。MongoDB 以 5.27 GB 磁盘数据(WiredTiger snappy)承载 15.21 GB 逻辑 BSON。

DuckDB 与 MongoDB 最紧凑(表 3),DuckDB 靠列式编码、MongoDB 靠压缩;即便未压缩,MongoDB 的 15.21 GB BSON 也不比关系型引擎的原始表更大。PostgreSQL 与 SQLite 在磁盘上要大上数倍。

3.2 检索操作

导航读取(点查、子节点、取内容)在除 DuckDB 外的每个引擎上都在亚毫秒级,DuckDB 并非为 OLTP 点读而生。在这些读取上 MongoDB 每次调用约比 PostgreSQL 慢 3 倍(表 4),其中一部分是每次调用的客户端开销,但在处处远低于一毫秒的量级上,这点差距没有实际代价;优化空间不在这里。唯一有代价的是 get_subtree,而且差距在尾部最大:一棵大子树返回数万个文档,MongoDB 的 P95 达到秒级,其它则保持在约 150 ms 以下(图 1)。物化路径区间也胜过递归的 parent_id 遍历,所以 ConDB 应保留前者。

操作	MongoDB	PostgreSQL	DuckDB	SQLite
中数据集,P50				
点查 (get_node)	0.238	0.081	2.705	0.008
展开子节点 (get_children)	0.251	0.090	0.856	0.017
取子树 (get_subtree)	0.498	0.210	2.700	0.091
取内容 (get_entity)	0.228	0.071	2.750	0.007
大数据集,P50 / P95				

点查	0.270 / 0.32	0.087 / 0.09	5.59 / 8.97	0.012 / 0.014
展开子节点	0.282 / 0.32	0.099 / 0.14	4.28 / 5.05	0.023 / 0.032
取子树(约 36k 行)	30.9 / 2478	10.7 / 123	9.9 / 47	11.8 / 135
取内容	0.249 / 0.32	0.081 / 0.09	5.21 / 8.35	0.012 / 0.013

表 4:检索读取延迟(ms,越低越好)。

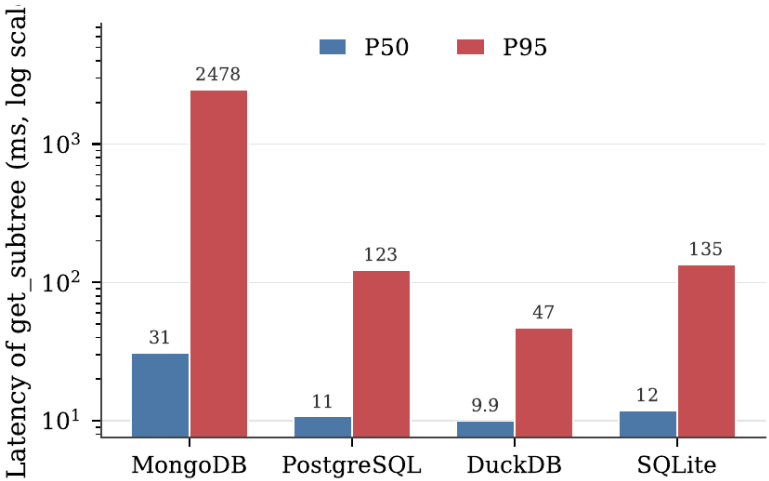


图 1:大数据集上的 get_subtree 延迟(对数刻度)。中位数彼此相差不大;MongoDB 的 P95 是异常点,达 2.5 s。

3.3 写入路径

在写入路径上(表 5),PostgreSQL 取得两个服务端引擎中更低的更新延迟,但每次 jsonb_set 都为该行生成新版本、留下一条待 VACUUM 的死元组。WiredTiger 同样是 MVCC,却会自动回收过期的文档版本,所以高更新负载不会累积膨胀;它实测的磁盘增量很小但有噪声,因为 WiredTiger 以检查点块为粒度报告存储。DuckDB 不适合 OLTP 写入。

引擎	更新 P50	更新 ops/s	插入 ops/s	膨胀
MongoDB	0.242	4,050	5,586	0 MB
PostgreSQL (JSONB)	0.123	7,856	8,732	7 MB
DuckDB	2.966	332	450	7 MB
SQLite	0.017	40,442	21,815	0 MB

表 5:写入路径(中数据集):5,000 次字段更新,2,000 次增量插入。“膨胀”指更新带来的磁盘增长。

3.4 并发

单次调用延迟低估了服务端引擎。用一组不断增长的独立 OS 进程(无 GIL 瓶颈)驱动点查时,两个服务端引擎都随客户端增加而提升吞吐,与嵌入式引擎的差距远比单次调用数字所暗示的要小(图 2)。MongoDB 在整个区间稳步上升;PostgreSQL 在约 16 个客户端附近见顶;SQLite 绝对值最快,但最早见顶(头几个客户端内),随后随进程争用其单一文件而退化。重复运行会让峰值的位置与高度有所移动,但形状不变。

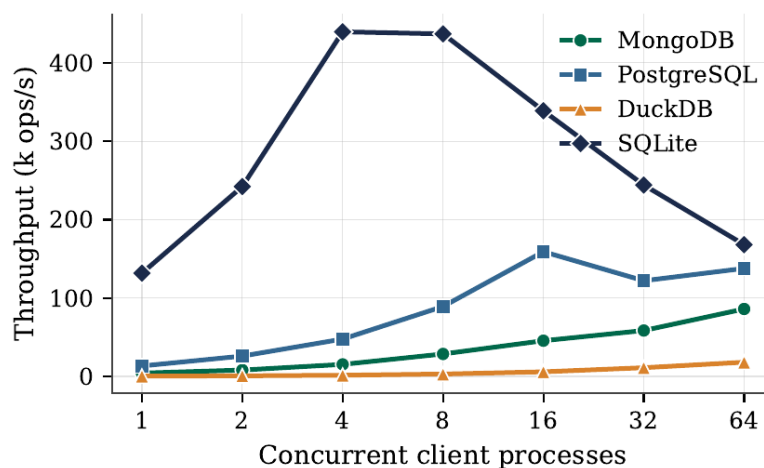


图 2:点查吞吐随并发客户端进程数的变化(中数据集)。

3.5 多租户、批量读取与删除

把检索器的调用映射到存储,会浮现出前几节遗漏的三个操作:多树共存于一个存储时每次查询的租户过滤、导航步发出的批量 id 列表读取,以及整棵树的删除。我们在把中数据集树复制成 25 棵共存树(单一存储内 1,771,075 行)上测量它们,每次读取都按 tree_id 过滤,由以 tree_id 打头的复合索引服务。

多租户选择性。 25 棵树同处一个集合时,以 tree_id 打头的索引让每次读取都贴近其单树延迟(表 6):代价随索引深度变化,而非随共存树的数量。对一个大 25 倍的存储,MongoDB 在点查上约升 1.5 倍、在取子树上约升 2.6 倍(对照表 4),没有扫描爆炸,且 MongoDB 与 PostgreSQL 的排序不变。

操作	MongoDB	PostgreSQL	DuckDB	SQLite
点查	0.345	0.124	3.772	0.008
展开子节点	0.372	0.147	3.073	0.014
取子树	1.273	0.312	4.605	0.069

表 6:25 棵树共存于单一存储、各按 tree_id 过滤的每次查询读取(P50,ms)。

批量读取。 导航步返回一组候选节点 id,因此检索器一次取多个节点。一次批量读取(MongoDB 用 \$in、关系型引擎用 IN)在每个引擎上都胜过它所替代的点查循环,且差距随批量增大而扩大,因为在服务端引擎上,循环的每一步都是一次往返。在 200 个 id 时,MongoDB 批量 4.6 ms,而循环 273 ms,差 60 倍;PostgreSQL 是 1.4 ms 对 29 ms(表 7)。MongoDB 受益最大,因为批量摊薄了它在单次读取上所付的每次调用客户端开销;进程内的 SQLite 没有往返可摊,只快约 2 倍。检索器的做法应是用一次批量读取解析候选集,而不是循环。

引擎	10	50	200	循环 / 200
MongoDB	0.46	1.56	4.56	273
PostgreSQL (JSONB)	0.27	0.78	1.40	29
DuckDB	19.6	22.2	39.1	790
SQLite	0.03	0.13	0.50	0.97

表 7:批量 id 列表读取在 10/50/200 个 id 时,对照它所替代的 200-id 点查循环(P50,ms/批)。前三列为批量读取。

删除。 删掉一棵树(此处为一个租户、1,771,075 行中的 70,843 行)是生命周期操作,不属于检索。删除本身在每个引擎上都快(不到四秒),但仅靠删除没有一个引擎会归还磁盘空间(表 8)。SQLite(VACUUM)与 PostgreSQL(VACUUM FULL)会通过一次全量、加锁的重写收回被删树的占用,耗时 10 至 17 秒;MongoDB 的 compact 与 DuckDB 对单租户删除并未让文件明显缩小,这是 WiredTiger 删除后不向操作系统归还空间的既定行为。对 MongoDB 而言这与写入路径对称:WiredTiger

在更新下不背负 vacuum 债,代价则是一次性批量删除不会自动缩容。WiredTiger 以检查点块为粒度报告 storageSize,所以几十 MB 以下的变化是报告粒度,而非已回收的空间。

引擎	删除 行/秒	回收	回收耗时	磁盘(前 → 后)
PostgreSQL (JSONB)	157,578	VACUUM FULL	17.3 s	2.77 → 2.66 GB
SQLite	141,842	VACUUM	10.6 s	2.56 → 2.46 GB
MongoDB	60,680	compact	0.1 s	911 MB,无缩容
DuckDB	20,171	checkpoint	—	790 MB,无缩容

表 8:整树删除与空间回收(在 25 棵树中删除其一)。

这三项都不改变本报告的结论:MongoDB 在大规模下守住租户过滤、在批量读取上完胜,删除上只在一个检索从不触发的、不加锁的缩容上落后。

4 为什么 MongoDB 在 get_subtree 上落后

原因是结构性的,而非配置失误。MongoDB 的读取单位是整份文档。一个二级索引(这里在 path 上)是一棵只存被索引字段与记录 id 的独立 B 树;要返回任何其它字段,引擎都必须取出完整的 BSON 文档再对其投影。一次点查就是一次这样的取出;一棵大子树则是数万次,工作负载中最大的那棵远不止此。数据常驻缓存(且 WiredTiger 缓存中的页是未压缩存放的),所以代价既非磁盘 I/O 也非解压:时间花在每文档的工作上 在 B 树里定位每条记录、物化包含 text 负载在内的整份文档,只为投影出一个节点 id,再把数万条结果经游标分批传出。乘以子树大小,这份每文档开销就是那条数秒的尾巴。其它操作只触达少量文档,故保持快速。

尾部还是超线性的。所有引擎回答同样的 200 个子树查询,故其延迟分布的离散程度可比。PostgreSQL 与 SQLite 的 P95/P50 比值约为 11 倍,跟随子树大小的离散;MongoDB 则坐在 80 倍,说明在最大的那些子树上,它每文档的代价本身也在上升,叠加了逐文档模型。

行式与列式引擎在每个节点上做的工更少。行存访问每一行匹配,却只物化被请求的列;宽 text 留在行里未被读取(PostgreSQL 甚至不解压该 JSONB datum,因为 node_id 是独立的列)。列存只以向量化批次读取被请求的那一列。两者还能更进一步 用一个能仅凭索引回答区间的覆盖索引(下文优化 1)。所有这些每行做的工都远少于一次每文档的取出。

这是文档模型的代价,也是 MongoDB 在别处取胜的原因:把一个节点的所有字段存在一起,让“取出整个实体”很快(点查与子节点读取)、让就地字段更新便宜(写入路径)、让布局利于分片(并发)、让磁盘形态紧凑(存储)。MongoDB 为另一种访问模式而优化,而 PageIndex 工作负载在除一个操作之外的每个操作上都顺着它。

5 优化方案

目标是 get_subtree;其余都不需要改。以下全是 MongoDB 社区版上的索引与 schema 设计,不改源码。

- 1. 覆盖索引 建在 path 加上树视图要渲染的字段(node_id、title、summary)上,投影限定为这些字段加 _id:0。get_subtree 于是仅凭索引运行,永不取出每个节点庞大的 text 正文,而那正是尾部之源。杠杆最高、纯配置;代价是更大的索引。
- 2. 拆分大 text 字段到独立集合(结构与摘要留在节点中,内容按节点 id 另存)。更小的热文档让每次取出和工作集都更省。

3. **嵌套文档子树模型。** 把有界子树存成单份嵌套文档,使一次子树读取从 36,000 次取出变为寥寥几次。一棵全文子树会数倍超过 MongoDB 的 16 MB 文档上限,所以它与第 2 项的拆 text 配合:几层深的“结构+摘要”分桶能稳稳处在上限之下。潜在收益最大;代价是每节点更新变成对外层文档的重写。

4. **优先用 get_subtree 的物化路径区间形式,而非递归的 parent_id 遍历。**

点查、子节点与内容读取、写入路径、并发与存储已经是 MongoDB 的强项,无需改动。对整个语料的临时分析(PageIndex 检索并不执行)需要的是一个列式旁路存储,而非改动 MongoDB。

6 优化讨论的开放问题

本报告确立了优化精力该投向何处。以下几点仍开放,我们把它们作为后续讨论的议程提出。

1. **覆盖索引实战中真能赢吗?** 1024 字节索引键上限已在 MongoDB 4.2 移除,所以带 title 和 summary 的索引是合法的。但覆盖查询之所以快,是因为索引键通常很小且常驻缓存;把数 KB 的摘要作为索引键,会让索引膨胀到接近集合本身,而我们没见过键这么宽的已公开实测。第一个实验就是在大数据集上做优化 1 的前后对比,经 explain 确认计划被覆盖(投影必须排除 _id),并观察索引大小与缓存压力。

2. **嵌套子树模型的分桶尺寸。** 按实测每节点约 1.5 kB 逻辑 BSON,基准里的大子树(约 36k 节点)作为单份文档约 55 MB,数倍于 16 MB 上限。几层深的“结构+摘要”分桶在这个合成语料上轻松够小,但真实摘要参差不齐;分桶深度应按最坏情况的真实文档设定,而非合成平均值。

3. **Nested Sets 作为替代。** MongoDB 文档把 Nested Sets 模式列为静态树的高效子树读取方案。PageIndex 树在摄入后实质上是写一次的,正好契合;冲突在增量插入路径,Nested Sets 会把它变成一次重编号。ConDB 的插入速率是否相容,是本次讨论的一个问题。

4. **内存受限场景。** 基准主机把每个引擎的数据都放在内存里。优化 2 的拆 text 主要靠缩小工作集获益,而这是常驻缓存的测量无法显示的。把服务端内存压到低于数据规模再跑一遍就能测到它,那也正是 MongoDB 磁盘紧凑性应当兑现的场景。

5. **优化方案如何改变批量读取与删除。** 这两个操作是在今日“一文档一节点”的 schema 上测的。拆 text(优化 2)与嵌套子树模型(优化 3)会同时改变两者:拆分会让需要内容的批量读取触及两个集合,嵌套模型会把整树删除变成更少、更大的文档删除。schema 改动后重新测量它们,属于优化工作本身。

7 结论

对 PageIndex 检索工作负载而言,MongoDB 在点查与子节点读取、内容获取、并发上都有竞争力,更新下不累积死元组膨胀(无 vacuum 债),且占用紧凑。它唯一的弱点是大 get_subtree 调用的尾延迟 逐份读取整文档的直接后果。这道差距能在社区版上用覆盖索引、拆出的内容集合与嵌套子树模型合上,同时保住 MongoDB 在写入、并发与占用上的优势。优化精力应只针对 get_subtree,别无其它。